



# Basics of ROS

Summer School

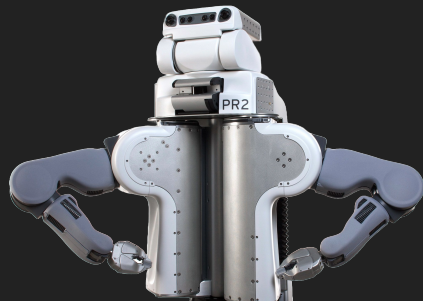
Soham and Tarun

June 10 2025

# History and Rationale

## 2 Motivation behind this Lecture

- Enter Keenan Wyrobek and Eric Berger, Stanford Students
- **Origins:** Stanford AI Lab, 2007–2009 (STAIR project & PR1)
- Willow Garage adoption (late 2007); first public ROS commit on November 7, 2007
- **Philosophy:** “Don’t reinvent the wheel” — leverage community packages/tools
- Vast ecosystem of packages, tools, and active community engagement

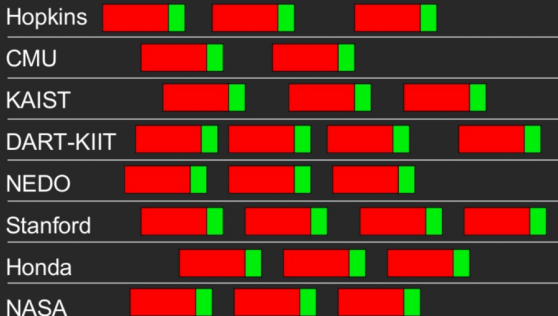
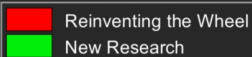


**Figure 1:** PR2—the flagship mobile manipulator

# Don't Reinvent the Wheel

## 2 Motivation behind this Lecture

### Enough of This



# And thus the community grew

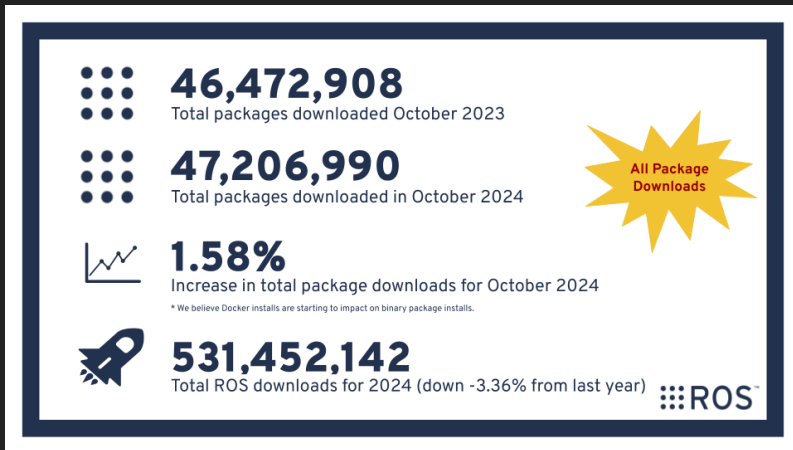
## 2 Motivation behind this Lecture





# Where are we now?

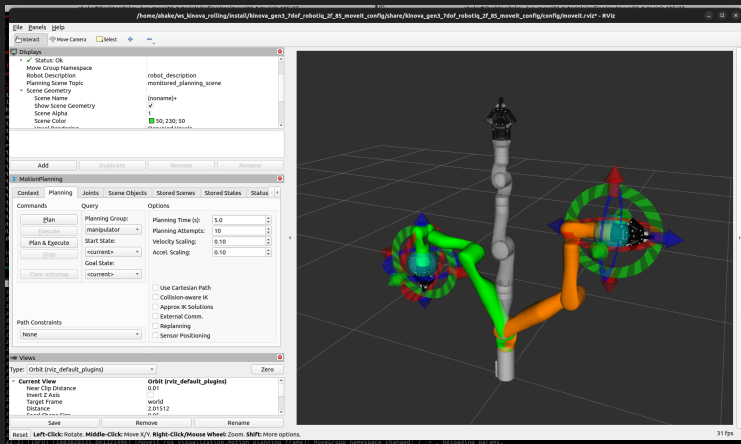
## 2 Motivation behind this Lecture



# RViz: Visualization Tool

## 4 ROS Components

- RViz is a 3D visualizer for the Robot Operating System (ROS) framework.



# Gazebo: Physics Simulation

## 4 ROS Components

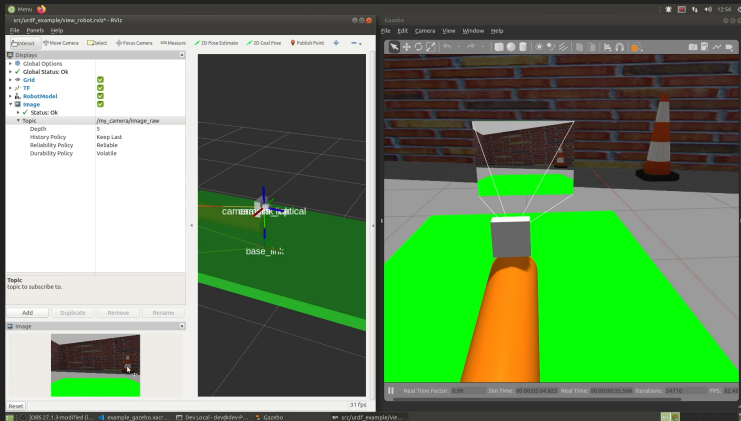
- Gazebo is a free, open-source robot simulation environment maintained by Open Robotics. With Gazebo, we can create a virtual "world", and load simulated versions of our robots into it. Simulated sensors can detect the environment, and publish the data to the same ROS topics that real sensors would, allowing easy testing of algorithms. Then, forces can be applied to the simulated actuators on the robot, taking into account things like friction.



# Gazebo to Hardware Transition

## 4 ROS Components

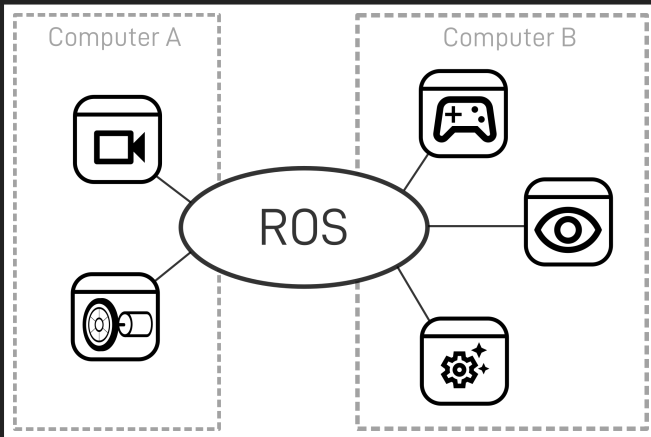
- Simulated sensor and actuator data mirrors real-world hardware.



# Nodes

## 5 Basic ROS Concepts

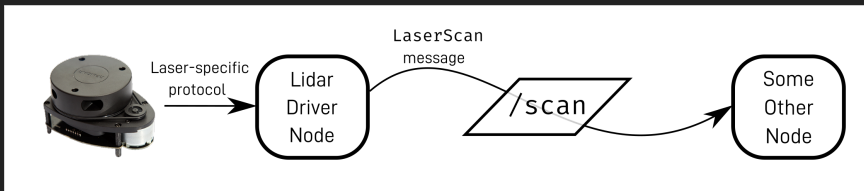
- A ROS system is made up of a bunch of smaller programs that all run at once and talk to each other. Each of these programs is called a node.



# Topics and Messages

## 5 Basic ROS Concepts

- The way ROS nodes communicate with each other is with topics and messages. A topic is a named location that one (or occasionally multiple) nodes can publish a message to. These nodes are called publishers, and other nodes (called subscribers) can subscribe to the topic to receive them.



# What can ROS do and what can it not do?

## ROS CAN DO THIS!!!

- Make the different sensors and actuators in your robot **communicate**
- **Execute** different pieces of code that runs your robot
- **Share information** between code and hardware
- Provide **standards and conventions** for you to develop custom stuff
- **Handle coordinate transforms** for various parts of your robot
- **Visualise** what your robot is doing

## ROS CANNOT DO THIS!!!

- **Allocate resources** like RAM and CPU
- **Manage users** like an OS
- Provide autonomy and perception **out-of-the-box**. You need either other people's libraries and works, or need to write your own.

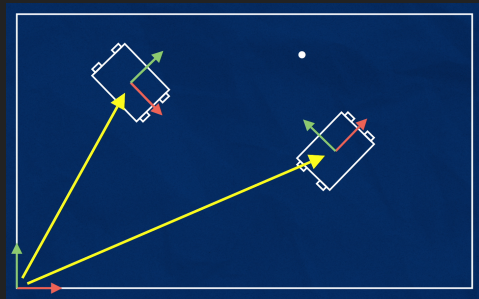
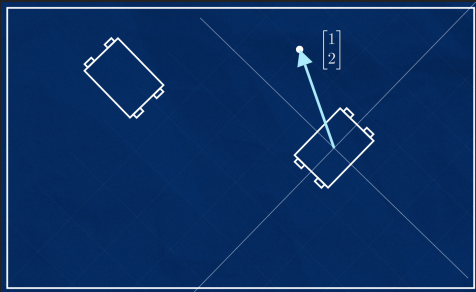
Robot Operating System  
is not an  
Operating System!!

# The Transform System (tf2)

4 ROS Components

## Why Transforms?

- They are a mathematical tool to take points or measurements that are represented from one point of view, and represent them in a different point of view that is more useful.

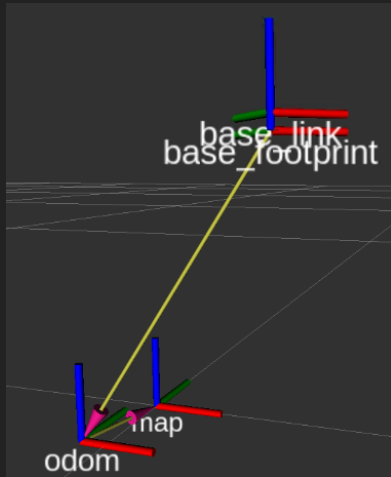




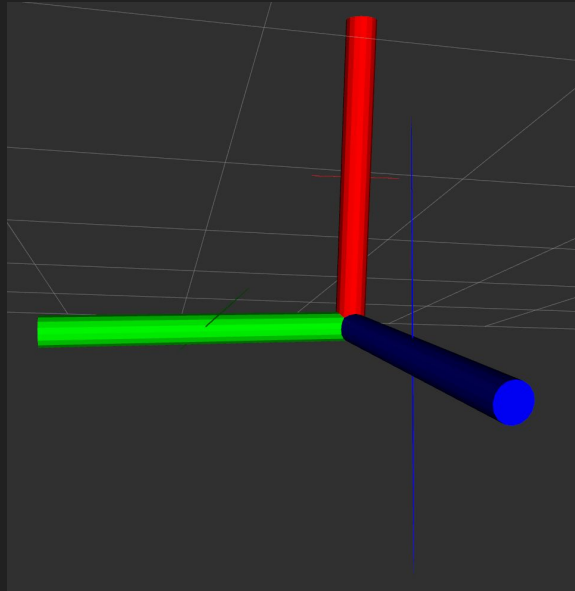
# Transforms in RVIZ

## 4 ROS Components

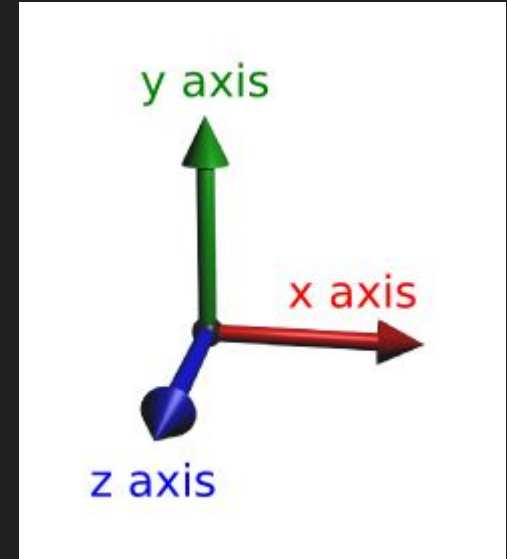
- In ROS 2, RViz (the ROS visualization tool) is called `rviz2`, which can display all kinds of different data. To display TF data, we click the "Add" button in the bottom-left corner, and select "TF".
- We can also configure the data displayed in RViz using the left panel.



# Axes Representation in RViz



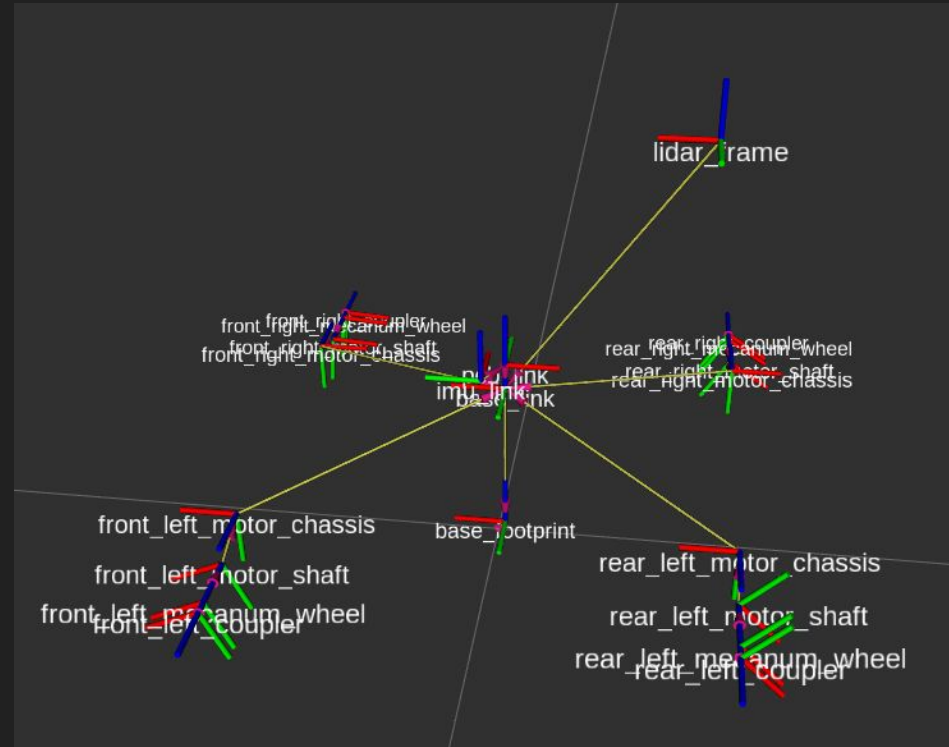
|   |   |
|---|---|
| R | X |
| G | Y |
| B | Z |



# Example TF Tree (AMR)

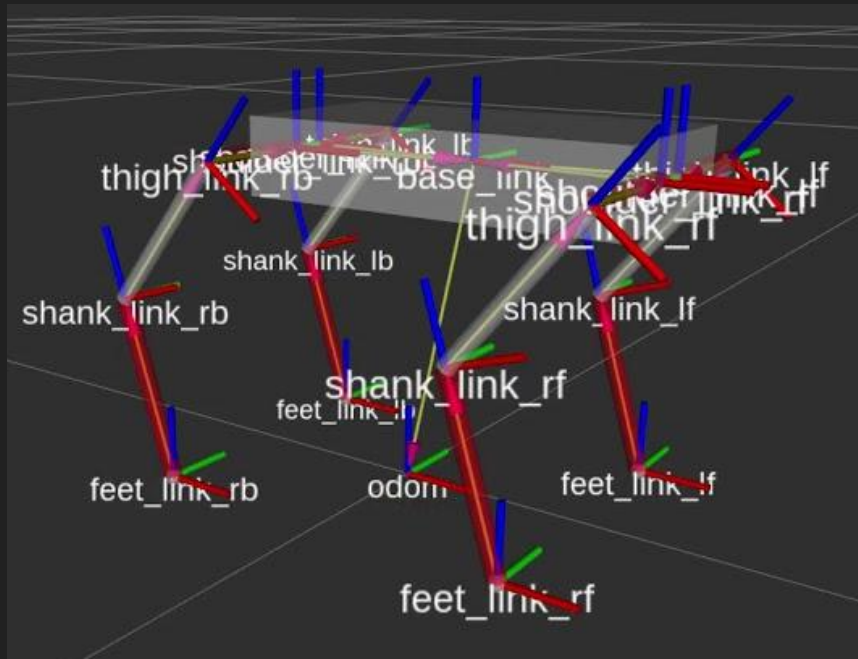


ROBOT

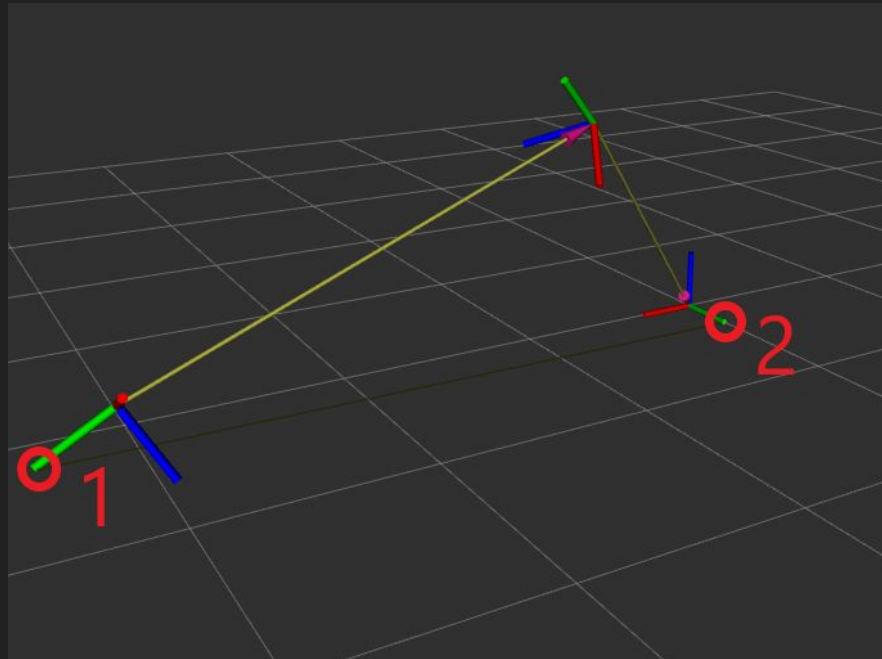


TF TREE

# Some more TF Trees



QUADRAPED

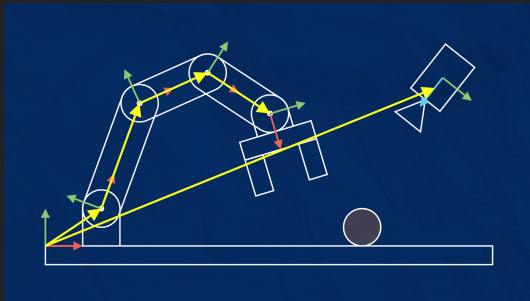


MANIPULATOR

# Broadcasting a Moving Robot

## 4 ROS Components

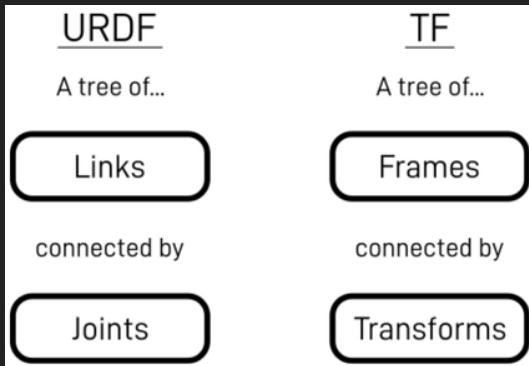
- The first step in doing this is to make sure we have a URDF file for our robot, which is a kind of configuration file where we can specify all sorts of physical characteristics of the robot's components, such as their size, shape, colour, and more.



# URDF (Unified Robot Description Format)

## 4 ROS Components

- In URDF, a robot is made up of a series of rigid components called links, and the links are joined together in a parent-child tree structure, where the relationships between different links are called joints.



# URDF (Unified Robot Description Format)

## 4 ROS Components

- The `/robot_state_publisher` which can take in a URDF file and automatically broadcast all the transforms from it. It will also publish the full contents of the URDF file to the topic `/robot_description` so that any other nodes that need it can all use the same file. URDF files are written in XML.

○ ○ ○

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="robot">

  <xacro:arg name="sim_mode" default="false"/>

  <xacro:include filename="robot_core.xacro" />

  <xacro:if value="$(arg use_ros2_control)">
    <xacro:include filename="ros2_control.xacro" />
  </xacro:if>
  <xacro:unless value="$(arg use_ros2_control)">
    <xacro:include filename="gazebo_control.xacro" />
  </xacro:unless>
  <xacro:include filename="lidar.xacro" />
  <xacro:include filename="camera.xacro" />

</robot>
```



# Launch Files

## 5 Basic ROS Concepts

### Transforms in ROS

- When we're working on a ROS project we will often be running the same nodes over and over again with the same parameters and remappings. This becomes pretty tedious as you need to remember to open all the terminals and get all the parameters right, then stop them all when you're done.

○ ○ ○

```
from launch import LaunchDescription
from launch_ros.actions import Node

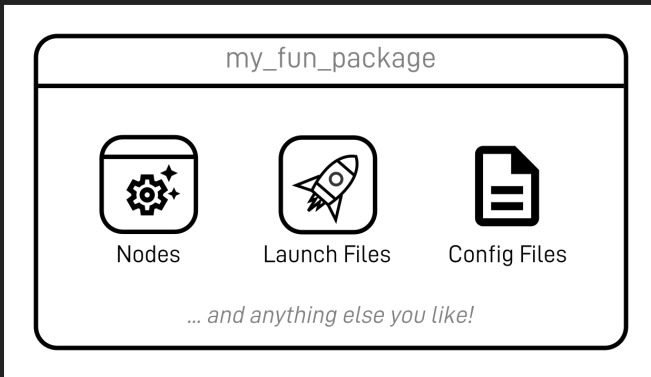
def generate_launch_description():
    return LaunchDescription([
        Node(
            package='rplidar_ros',
            executable='rplidar_node',
            parameters=[{
                'serial_port': '/dev/ttyUSB0'
            }],
            remappings=[
                ('/scan', '/scan_1')
            ]
        )
    ])
```



# ROS Packages

## 5 Basic ROS Concepts

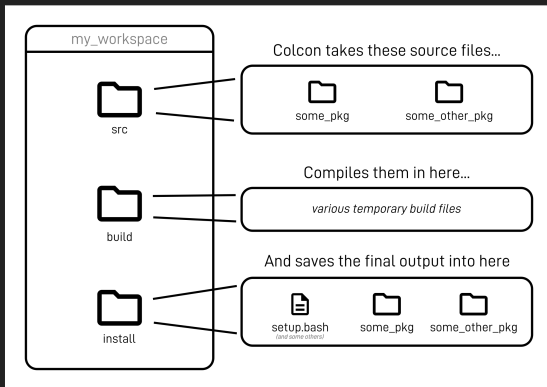
- Packages are the basic unit of organization in ROS, grouping related nodes, configuration files, models, or libraries together for reuse and modular development.



# ROS Workspaces

## 5 Basic ROS Concepts

- Packages are the basic unit of organization in ROS, grouping related nodes, configuration files, models, or libraries together for reuse and modular development.



# Docker

# What's the laziest way to solve this problem?

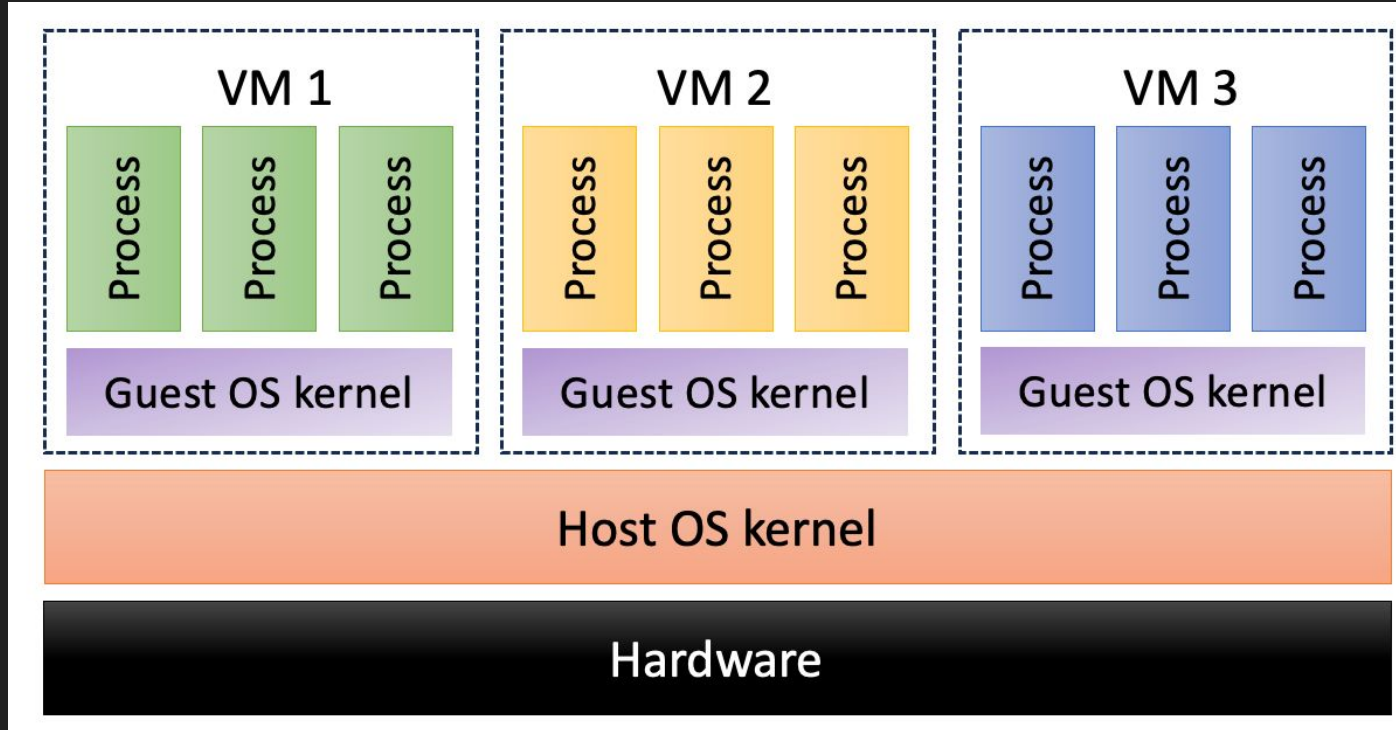
Imagine you are an IT administrator at a poorly-funded start-up. You have to distribute laptops with some very specific software (10 different apps, all running on top of Windows 11) to 1000 people. Each laptop also has to have network login for your company built-in.

You are provided with 1000 brand-new blank laptops (no OS, no apps, no files), a very fast internet connection. And a device that can upload whatever you want to a laptop's SSD super fast.

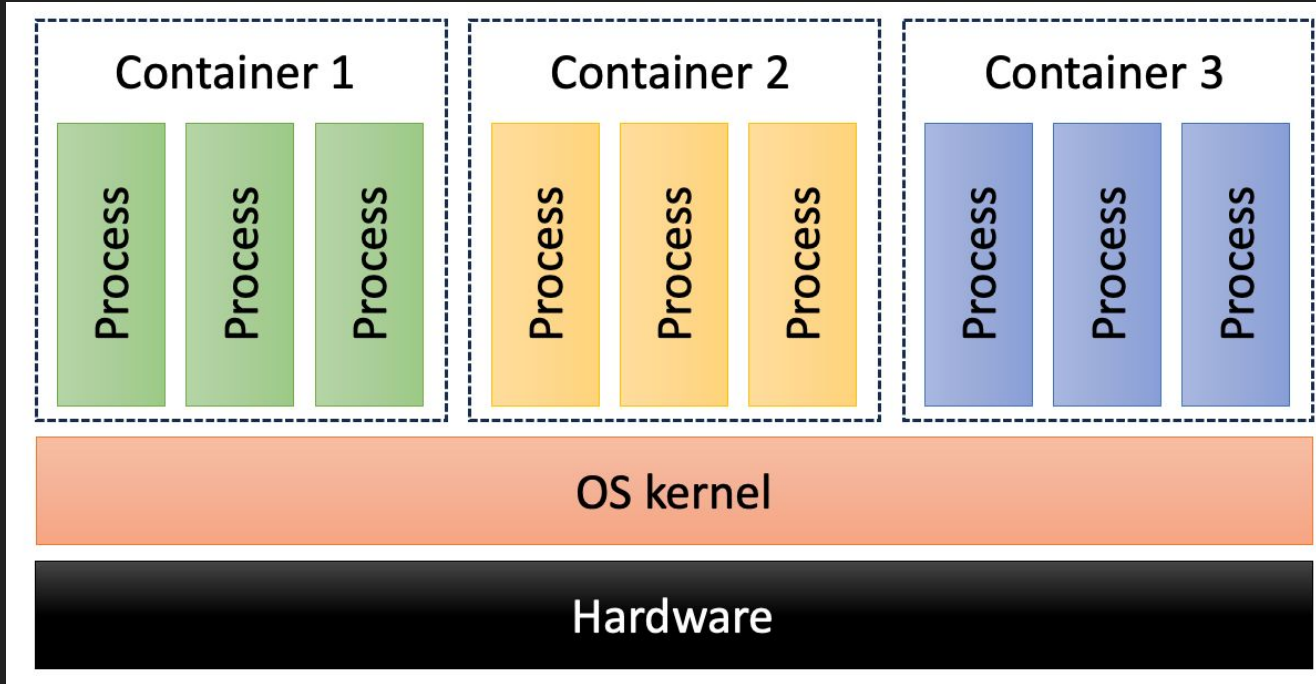
How will you do it?

You may not pass on the burden to another person (neither a colleague, nor the recipients of the laptops, nor a shady contractor).

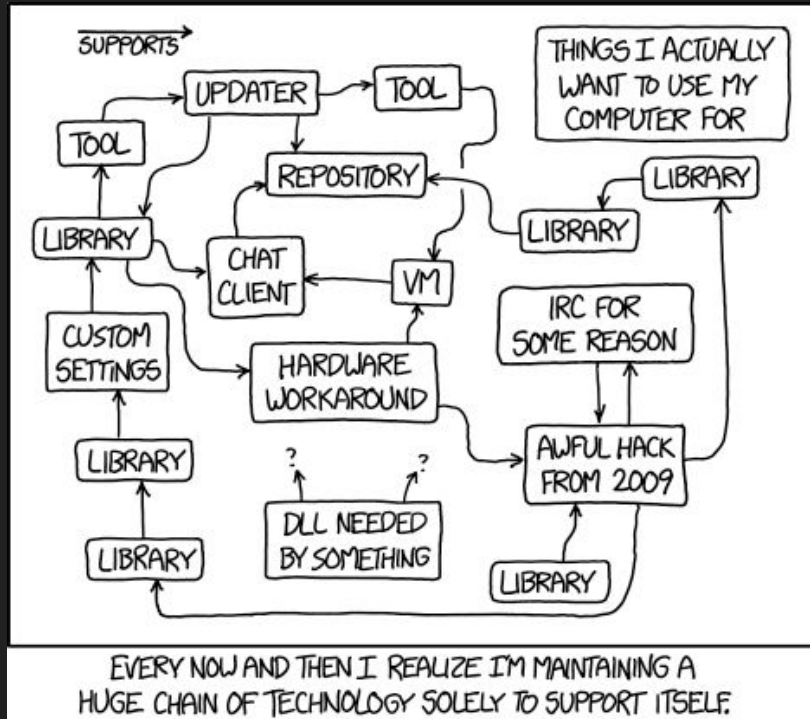
# Now how to run multiple customised images at once?



# Containers: more efficient approach



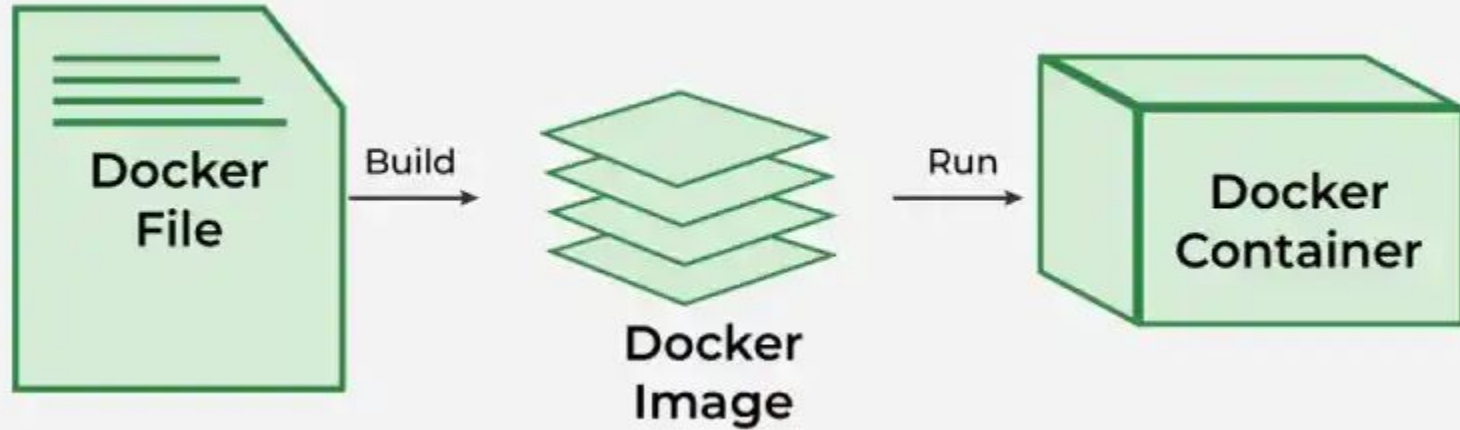
# Why use Docker?



Mainly to save money on headache medication  
(although it may initially seem like the opposite).

- **Repeatability:** If you successfully deploy something in a Docker Container, you can just copy it somewhere else.
- **Isolation:** ROS often messes with the computer it is installed on. So we let it mess around inside Docker rather than in our main PC.
- **Transparency:** You can see exactly what is there in a Dockerfile

# Basics of Docker





# Assignments for tomorrow

## GET FAMILIAR WITH DOCKER

1. Download Docker
2. Create a Dockerfile starting from the official Python image, and install numpy and matplotlib in it. Make sure your installations happen inside the Dockerfile.
3. Create an image from this and spin up a container.
4. Take any old project that produces a graphical output (matplotlib) and make the plot visible to you. You may not save it as an image or use iPython/Jupyter; you must use the `matplotlib.pyplot.show()` function.

## SET UP A ROS WORKSPACE

1. Setup your Docker installation using the NVIDIA Container Toolkit.
2. Pull this image:  
[ghcr.io/soham2560/humble-harmonic:latest](https://ghcr.io/soham2560/humble-harmonic:latest)
3. Recursively clone this GitHub repo:  
<https://github.com/rtarun1/HepoBot>